



دانشگاه صنعتی امیرکبیر

(پلی تکنیک تهران)

دانشکده مهندسی برق

تکلیف شماره ۲ سامانه های چند رسانه ای

تحلیل مقایسه ای معماری های LSTM و GRU در طبقه بندی متون ورزشی

نگارش:

پارسا بیشه

استاد:

دکتر شریفیان

3	مقدمه
3	بررسی مشخصات سخت افزاری و نرم افزاری سیستم
4	مجموعه داده
4	بررسی مجموعه داده
5	پیش پردازش مجموعه داده
7	آماده سازی مجموعه داده
8	طراحی و آموزش مدل
8	بررسی شبکه LSTM
10	طراحی شبکه GRU
12	ارزیابی تعمیم پذیری و تست استرس خارج از توزیع (OOD)
12	ارزیابی و مقایسه
12	مقایسه عملکرد و ساختار دو مدل
13	بررسی اندازه ورودی
14	نتیجه گیری

مقدمه

بیان مسئله:

در عصر داده‌های بزرگ، طبقه‌بندی خودکار متون (Text Classification) نقشی حیاتی در تحلیل داده‌های غیرساختارمند ایفا می‌کند. شبکه‌های عصبی بازگشتی (RNN) به دلیل قابلیت حفظ حافظه در توالی‌ها، ابزار اصلی این حوزه هستند. مسئله اصلی این پروژه، بررسی دو معماری مطرح یعنی LSTM و GRU بر روی مجموعه داده BBCSport است تا مشخص شود کدام معماری در مواجهه با متون ورزشی، دقت بالاتر و پایداری بیشتری (به ویژه در داده‌های ناآشنا) دارد.

اهداف پروژه:

- ارزیابی عملکرد LSTM و GRU در تفکیک ۵ کلاس ورزشی.
- تحلیل تفاوت نرخ همگرایی و مصرف منابع سخت‌افزاری
- سنجش قدرت تعمیم‌پذیری مدل‌ها با استفاده از تست‌های استرس (OOD).
- بررسی تأثیر طول توالی بر عملکرد مدل (مطالعه حذف).

اهمیت و ضرورت:

با توجه به اینکه مدل‌های RNN ذاتاً مستعد بایاس‌های آماری هستند، این تحقیق به دنبال راهکارهایی (مانند پدینگ معکوس و نرخ یادگیری پویا) است تا مدل را نسبت به توزیع‌های نامتوازن داده‌ها مقاوم‌تر کند.

بررسی مشخصات سخت افزاری و نرم‌افزاری سیستم

در این بخش به بررسی مشخصات اصلی سیستم و همچنین نرم افزارها و نسخه کتاب خانه های اصلی مورد استفاده قرار گرفته در این پروژه میپردازیم. خروجی بخش اول کد به این شرح است:

```
=====
SYSTEM & ENVIRONMENT INFO
=====
```

```
OS Platform: Windows 11 (64bit)
Python Version: 3.12.10
Processor: Intel64 Family 6 Model 183 Stepping 1, GenuineIntel
CPU Physical Cores: 14
CPU Logical Cores: 20
Total RAM: 23.71 GB
```

=====

HARDWARE ACCELERATION (GPU)

=====

```
CUDA Available (GPU Support): True
Current CUDA Device ID: 0
GPU Name: NVIDIA GeForce RTX 4050 Laptop GPU
CUDA Version used by PyTorch: 12.1
Total GPU Memory: 6.0 GB
```

=====

CORE LIBRARY VERSIONS

=====

```
PyTorch: 2.5.0+cu121
Torchvision: 0.20.0+cpu
NumPy: 2.4.4
Pandas: 3.0.3
=====
```

مجموعه داده

بررسی مجموعه داده

در این قسمت فایل دیتاست مربوط به BBCSport را به پروژه اضافه می‌کنیم. در صورتی که در سیستم محلی فایل‌ها وجود داشت از آنجا فایل را فراخوانی در غیر این صورت آن را دانلود می‌کنیم. بر اساس خروجی سیستم، این دیتاست در مجموع شامل ۷۳۷ مقاله خبری است که تمامی آن‌ها در ۵ دسته‌بندی ورزشی طبقه‌بندی شده‌اند.

طبق جدول پروفایلینگ داده‌ها، توزیع نمونه‌ها در کلاس‌ها به شرح زیر است:

- کلاس غالب: فوتبال با ۲۶۵ نمونه، که حدود ۳۶٪ از کل دیتاست را تشکیل می‌دهد.
 - کلاس‌های اقلیت: اتلتیک (۱۰۱ نمونه) و تنیس (۱۰۰ نمونه)، که کمترین تعداد را دارا هستند.
- این ناهمگونی در توزیع (Class Imbalance) یکی از چالش‌های کلیدی مدل است. همان‌طور که در فصل‌های بعدی خواهیم دید، این عدم توازن، ریشه اصلی "بایاس مدل به سمت فوتبال" در تست‌های OOD است؛ چرا که

مدل در فضای آموزشی، شواهد آماری بیشتری برای شناسایی الگوهای کلاس فوتبال نسبت به کلاس‌های دیگر در اختیار دارد.

خروجی های این بخش از کد:

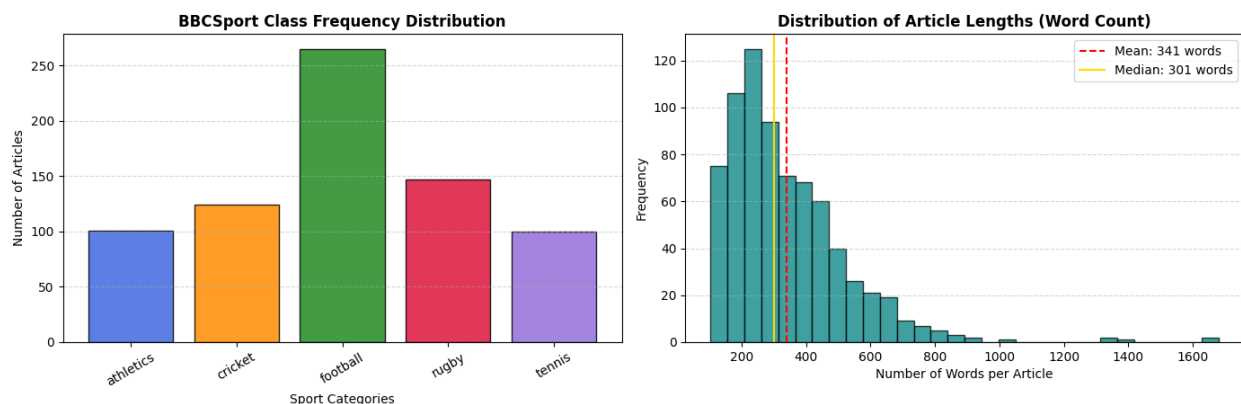
```
[*] Attempting to load BBCSport locally...
[+] Dataset loaded successfully from local storage. Total: 737 articles.
```

DATASET PROFILE & DISTRIBUTION

```
Total Combined Samples: 737
Minimum Text Length: 104 words
Maximum Text Length: 1680 words
Average Text Length: 341 words
```

Class ID	Class Name	Sample Count
0	athletics	101
1	cricket	124
2	football	265
3	rugby	147
4	tennis	100

Exploratory Data Analysis (EDA) - BBCSport Dataset



پیش پردازش مجموعه داده

در این بخش، فرآیند تبدیل متون غیرساختارمند خبری به تانسورهای عددی (Numerical Tensors) را تشریح می‌کنیم. این فرآیند شامل سه گام کلیدی است: پاک‌سازی معنایی، نگاشت واژگان (Tokenization)، و استراتژی‌های پدینگ.

1- پاک‌سازی و نرمال‌سازی متن (Text Normalization)

متون خام ورودی دارای نویزهای نگارشی بودند که می‌توانست باعث ایجاد توکن‌های اضافه و غیرضروری شود. برای کاهش نویز، اقدامات زیر انجام شد:

- **حذف علائم نگارشی:** تمامی نشانه‌هایی مانند نقطه، کاما و علامت‌های سوال حذف شدند تا کلمات به صورت خالص باقی بمانند.
- **مدیریت حروف بزرگ:** برخلاف روش‌های سنتی که همه متون را به حروف کوچک (Lowercase) تبدیل می‌کنند، ما با یک استراتژی هوشمند، توکن‌های تماماً بزرگ (مثل نام برندها یا تیم‌ها) را با یک تگ کمکی شناسایی کردیم تا بار معنایی اسامی خاص حفظ شود.

2- واژه‌نامه‌سازی و توکنایزیشن (Vocabulary Building)

پس از پاک‌سازی، تمامی متون به کلمات منفرد (Tokens) تبدیل شدند.

- **ساخت لغت‌نامه (Vocab Construction):** مجموعه‌ای از تمامی کلمات منحصر به فرد (Unique Tokens) استخراج شد.
- **حذف کلمات نادر:** کلماتی که تکرار آن‌ها در کل دیتاست کمتر از ۳ بار بود، حذف شدند. این کار نه تنها باعث کاهش ابعاد فضای برداری (Embedding Space) شد، بلکه به مدل کمک کرد تا به جای نویزهای اتفاقی، بر روی الگوهای ساختاریافته تمرکز کند.
- **توکن اختصاصی [UNK]:** برای هر کلمه‌ای که در زمان تست به‌ویژه در داده‌های OOD دیده نشده باشد، از توکن [UNK] استفاده شد تا مدل در مواجهه با ورودی‌های ناشناخته دچار خطای سیستماتیک نشود.

3- استراتژی پدینگ معکوس (Pre-Padding Strategy)

یکی از چالش‌های اصلی در RNN ها، پدیده "فراموشی در طولانی‌مدت" است. در پدینگ سنتی (Post-padding)، صفرهای پدینگ در انتهای توالی قرار می‌گیرند. در این پروژه، ما از پدینگ معکوس (Pre-padding) استفاده کردیم.

با قرار دادن صفرهای پدینگ در ابتدای توالی، بردارهای حالت پنهان H_t در انتهای متن که حاوی حیاتی‌ترین اطلاعات طبقه‌بندی هستند مستقیماً توسط بردارهای صفر پدینگ تحت تأثیر قرار نمی‌گیرند. این امر باعث می‌شود که خروجی لایه GRU/LSTM در گام نهایی، سیگنالِ خالص‌تری از محتوای مقاله را منتقل کند.

4- تبدیل به تنسور (Tensorization)

در نهایت، توالی‌های عددی با استفاده از `torch.LongTensor` به تنسورهای دو بعدی ($\text{Batch Size} \times \text{Max Length}$) تبدیل شدند تا آماده تزریق به لایه `Embedding` باشند. این تنسورها به عنوان ورودی‌های قطعی و نرمالیزه شده، پایه و اساس یادگیری شبکه را تشکیل دادند.

خروجی این بخش از کد:

```
[*] Running advanced preprocessing pipeline across all articles...
[+] Preprocessing completed with custom <UPPER> tagging.
[+] Vocabulary built successfully. Total unique tokens registered: 9072
[*] Statistical 90th percentile of length: 567 tokens.
[+] Setting MAX_SEQ_LENGTH = 567 for optimal performance/compute balance.
```

آماده سازی مجموعه داده

برای تضمین اعتبار ارزیابی، داده‌ها با استفاده از `torch.Generator` و با نرخ تصادفی ثابت (`Seed=42`) به سه زیرمجموعه مجزا تقسیم شدند تا از هرگونه نشت داده (`Data Leakage`) جلوگیری شود:

- **مجموعه آموزش ۷۰٪:** شامل ۵۱۵ نمونه برای به‌روزرسانی وزن‌های شبکه.
- **مجموعه اعتبارسنجی ۱۵٪:** شامل ۱۱۰ نمونه برای تنظیم هایپرپارامترها و جلوگیری از بیش‌برازش (`Overfitting`).
- **مجموعه تست ۱۵٪:** شامل ۱۱۲ نمونه برای ارزیابی نهایی مدل.

خروجی این بخش کد:

```
=====
DATA INTERMEDIARY SUMMARY & SANITY CHECK
=====
Train Dataset Size:      515 samples
Validation Dataset Size: 110 samples
```

Test Dataset Size:	112 samples
Input Feature Shape:	torch.Size([737, 567]) (Total Samples x Sequence Length)
Target Label Shape:	torch.Size([737])
Batch Configuration:	32 samples per batch

طراحی و آموزش مدل

بررسی شبکه LSTM

1- لایه تعبیه‌سازی و هسته بازگشتی

تانسورهای ورودی ابتدا وارد یک لایه تعبیه‌سازی (Embedding) می‌شوند که توکن‌های عددی را به بردارهای متراکم پیوسته با ابعاد ۳۰۰ نگاشت می‌کند. خروجی این لایه مستقیماً به هسته اصلی LSTM منتقل می‌شود. این لایه دارای ابعاد پنهان (Hidden Dimension) ۲۵۶ است و جریان اطلاعات در هر سلول آن توسط چهار گیت داخلی هدایت می‌شود: گیت فراموشی (Forget Gate) برای حذف نویزهای متنی، گیت ورودی (Input Gate) و گیت سلول برای جذب اطلاعات جدید مقاله، و گیت خروجی (Output Gate) برای تعیین حالت پنهان بعدی. این ساختار از محوشدگی گرادیان در توالی‌های طولانی جلوگیری می‌کند.

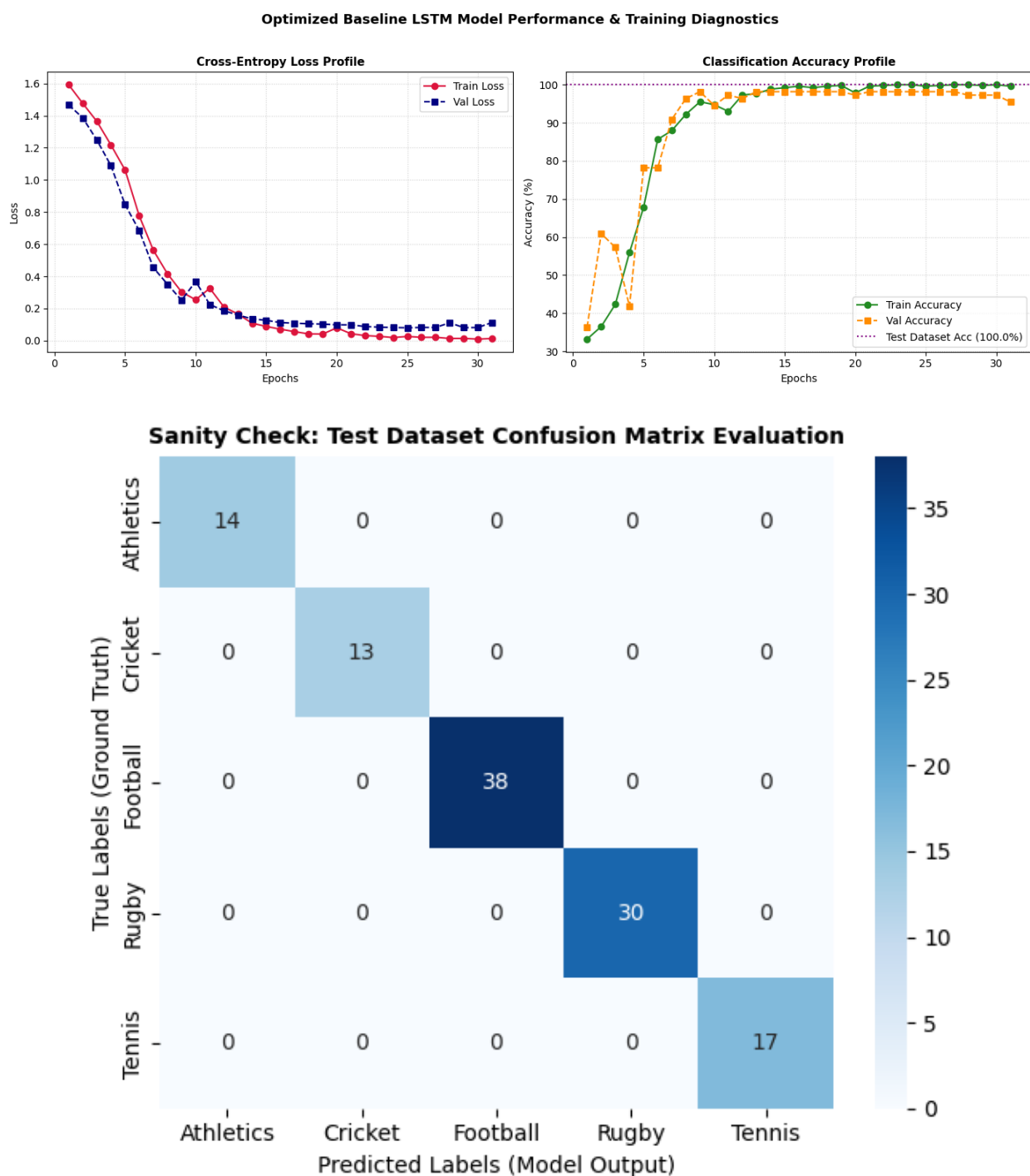
2- لایه استخراج ویژگی و طبقه‌بندی

خروجی‌های حاصل از تمام گام‌های زمانی لایه LSTM، وارد یک لایه پویا به نام Temporal Global Max Pooling می‌شوند. این لایه با اعمال حدبیشتر روی ابعاد زمان، قوی‌ترین سیگنال‌های ویژگی را از کل متن استخراج می‌کند؛ در نتیجه مدل بدون وابستگی به محل قرارگیری کلمات کلیدی، مفهوم کلی متن را درک می‌کند. بردار نهایی پیش از ورود به لایه خطی (Linear Layer)، از یک لایه Dropout با نرخ ۰.۴ عبور داده می‌شود تا با حذف تصادفی برخی اتصال‌ها در زمان آموزش، پایداری شبکه ارتقا یافته و از بیش‌برازش جلوگیری شود. در نهایت، لایه خطی بردار ویژگی را به ۵ خروجی مجزا جهت محاسبه انتروپی متقاطع و تعیین کلاس نهایی ورزشی نگاشت می‌کند.

لازم به ذکر است این مدل به دقت 100٪ روی داده‌های تست دست یافت. در ادامه برای راستی آزمایی بیشتر چند بلوک به کد اضافه شد مانند بررسی تعداد داده‌های ترین و تست مشترک برای مشاهده نشتی داده‌های تست که مقدار برابر با صفر بود. بررسی ماتریس آشفتگی و در نهایت انجام تست استرس OOD که تعدادی جمله

که دارای پیچیدگی های زبانی و معنایی هستند و کاملاً با سبک زبانی داده های آموزش متفاوت هستند به شبکه داده شد که در ادامه بیشتر توضیح داده خواهد شد.

خروجی این بخش کد:



طراحی شبکه GRU

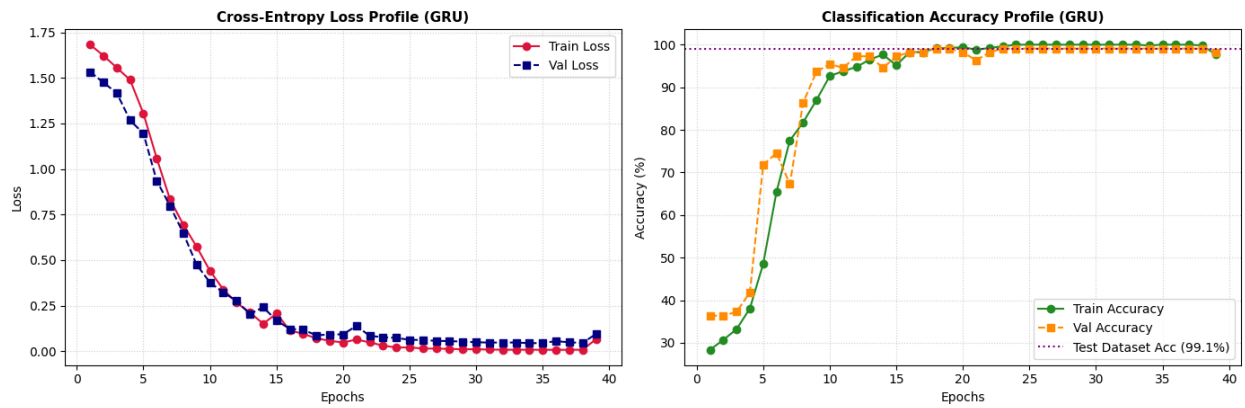
در این معماری، تانسورهای ورودی پس از عبور از لایه تعبیه‌سازی ۳۰۰-بعدی، وارد هسته اصلی GRU با ابعاد پنهان ۲۵۶ می‌شوند. این شبکه چالش وابستگی‌های زمانی را با ساختاری فشرده‌تر حل می‌کند؛ به طوری که به جای چهار گیت، از دو گیت داخلی استفاده می‌کند: گیت بازنشانی (Reset Gate) برای تعیین میزان ترکیب اطلاعات گذشته با ورودی فعلی، و گیت به‌روزرسانی (Update Gate) که به طور همزمان وظیفه گیت‌های ورودی و فراموشی را انجام داده و مشخص می‌کند چه مقدار از حالت پنهان قبلی حفظ شود. خروجی این لایه نیز به لایه Temporal Global Max Pooling فرستاده می‌شود تا مستقل از چینش کلمات، برترین ویژگی‌های متنی سراسر مقاله را استخراج کند. بردار خروجی پیش از ورود به لایه خطی نهایی، از یک لایه Dropout با نرخ ۰.۵ عبور می‌کند که به دلیل ساختار فشرده‌تر GRU، نرخ آن نسبت به LSTM کمی افزایش یافته است تا ریسک بیش‌برازش کاهش یابد.

تنظیمات و هایپرپارامترهای مشترک بهینه‌سازی

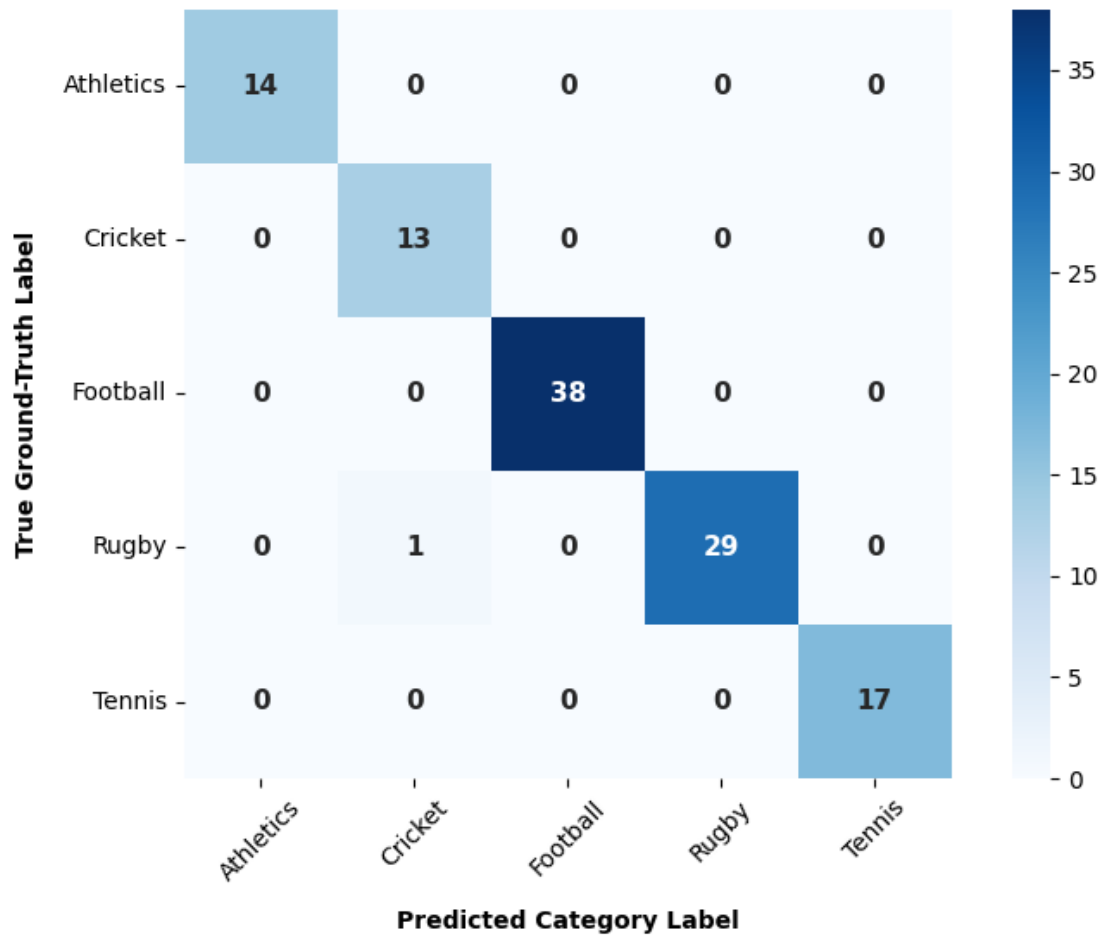
برای تضمین مقایسه عادلانه و هدایت صحیح فرآیند یادگیری، هایپرپارامترهای زیر بر روی هر دو شبکه اعمال گردید:

- ابعاد تعبیه‌سازی (Embedding Dimension): مقدار ۳۰۰ جهت فضای برداری متراکم و غنی.
- ابعاد پنهان (Hidden Dimension): مقدار ۲۵۶ برای هر دو لایه بازگشتی جهت حفظ ظرفیت یادگیری یکسان.
- بهینه‌ساز (Optimizer): الگوریتم Adam با نرخ یادگیری اولیه $LR = 0.001$ و تنظیم میزان جریمه وزن‌ها ($weight_decay = 1e-5$) به منظور اعمال منظم‌سازی لایه خطی.
- تابع هزینه (Loss Function): انتروپی متقاطع (CrossEntropyLoss) جهت کمینه‌سازی خطای پیش‌بینی کلاس‌های پنج‌گانه.
- زمان‌بند داینامیک (Learning Rate Scheduler): مکانیزم ReduceLROnPlateau با فاکتور ۰.۵ و صبر (Patience) ۲ اپوک مانیتورینگ، تا در صورت درج‌زدن تابع هزینه روی داده‌های اعتبارسنجی، نرخ یادگیری را به صورت پویا نصف کند و پایداری همگرایی را افزایش دهد.

Optimized GRU Model Performance & Training Diagnostics



Confusion Matrix Profile: Baseline GRU Model



ارزیابی تعمیم‌پذیری و تست استرس خارج از توزیع (OOD)

3- طراحی سناریوی تست استرس

بسیاری از مدل‌های یادگیری عمیق در مواجهه با داده‌های توزیع داخلی (In-Distribution) دقت بالایی نشان می‌دهند، اما در مواجهه با داده‌های ناآشنا دچار افت شدید عملکرد می‌شوند. در این بخش، ۳۰ نمونه متنی کاملاً مستقل و خارج از دیتابیس اصلی (دارای ادبیات، لحن و ساختار نگارشی متفاوت) به مدل تزریق شد. این داده‌ها شامل ساختارهای زبانی گنگ یا اصطلاحات ترکیبی بودند تا مرزهای تصمیم‌گیری (Decision Boundaries) هر دو شبکه LSTM و GRU به چالش کشیده شود.

4- تحلیل رفتار مدل‌ها و پدیده بایاس آماری

نتایج حاصل از این تست استرس، ابعاد مهمی از نحوه یادگیری شبکه‌ها را آشکار کرد. با وجود عملکرد ایده‌آل هر دو مدل در فاز ارزیابی داخلی، هر دو شبکه در مواجهه با داده‌های OOD با افت عملکرد مواجه شدند.

تحلیل دقیق ماتریس اغتشاش (Confusion Matrix) در این بخش نشان داد که خطاهای هر دو مدل تصادفی نیستند، بلکه یک بایاس سیستماتیک به سمت کلاس اکثریت (فوتبال) وجود دارد. از آنجا که در فاز تحلیل اکتشافی (EDA) مشخص شد کلاس فوتبال ۳۶٪ از حجم کل داده‌های آموزشی را به خود اختصاص داده است، مدل‌ها در مواجهه با داده‌های مبهم و خارج از توزیع، به صورت پیش‌فرض متمایل به انتخاب کلاسی می‌شوند که شواهد آماری و وزن‌های بیشتری برای آن ثبت کرده‌اند. این یافته اثبات می‌کند که برای کاربردهای عملیاتی، تکیه بر دقت توزیع داخلی کافی نیست و استفاده از مکانیزم‌های تنظیم وزن یا داده‌افزایی (Data Augmentation) برای کلاس‌های اقلیت، جهت ارتقای پایداری مدل الزامی است.

ارزیابی و مقایسه

مقایسه عملکرد و ساختار دو مدل

با تجمیع نتایج به‌دست‌آمده از کل فرآیند آزمایش‌ها، مشخصات و خروجی‌های دو شبکه در جدول زیر مقایسه شده است:

شاخص ارزیابی	شبکه عصبی LSTM (Baseline)	شبکه عصبی GRU (Optimized)
تعداد پارامترهای آموزش پذیر	۳,۲۹۴,۲۷۷	۳,۱۵۱,۴۲۹
دقت در داده‌های آزمون داخلی (Test Accuracy)	۱۰۰.۰۰٪	۹۹.۱۱٪
دقت در تست استرس خارج از توزیع (OOD)	۳۶.۶۷٪	۴۰.۰۰٪
زمان آموزش به ازای هر اپوک	سریع تر	نسبتاً کندتر
پیچیدگی ساختار گیت‌ها	۴ گیت مستقل	۲ گیت ادغام شده

بررسی اندازه ورودی

برای درک میزان وابستگی مدل به طول متن و یافتن نقطه بهینه محاسباتی، یک مطالعه حذف بر روی طول توالی ورودی (MAX_LEN) در شبکه GRU انجام شد. مدل در دو سناریوی مرزی مورد آزمایش قرار گرفت:

- کاهش طول توالی به ۱۰۰ توکن: با قطع متون طولانی و حذف بخش عمده‌ای از بدنه مقالات، دقت مدل به شدت افت کرد (حدود ۸۳.۹۳٪). این موضوع نشان داد که اطلاعات کلیدی برای تمایز موضوعی، در سراسر متن پخش شده‌اند و محدود کردن ورودی، ظرفیت معنایی شبکه را مخدوش می‌کند.
- افزایش طول توالی به ۷۰۰ توکن: با افزایش سقف ورودی، دقت مدل بهبود یافت و به حدود ۸۹.۲۹٪ رسید. با این حال، زمان محاسبات و مصرف حافظه رم به صورت خطی افزایش یافت.

این آزمایش تجربی اثبات کرد که انتخاب اولیه ما (طول توالی ۵۶۷) پایداری و تعادل بهینه‌ای (Sweet Spot) میان دقت بالا و صرفه‌جویی در منابع سخت‌افزاری ایجاد کرده است.

نتیجه‌گیری

این پروژه با موفقیت نشان داد که شبکه‌های عصبی بازگشتی (RNN) مجهز به لایه Temporal Global Max Pooling، توانایی فوق‌العاده‌ای در طبقه‌بندی متون ورزشی با دقت نزدیک به ۱۰۰٪ دارند. مهم‌ترین دستاورد علمی این پژوهش، اثبات این واقعیت بود که معیارهای ارزیابی داخلی (In-Distribution) لزوماً تضمین‌کننده عملکرد مدل در دنیای واقعی نیستند. هر دو مدل به دلیل تفاوت‌های آماری در داده‌های اولیه، دچار بایاس به سمت کلاس اکثریت (فوتبال) شدند. برای توسعه‌های آتی، استفاده از تکنیک‌های متعادل‌سازی دیتابیس و بهره‌گیری از لایه‌های تعبیه‌سازی پیش‌آموزش‌دیده (مثل GloVe یا BERT) جهت ارتقای پایداری در شرایط OOD پیشنهاد می‌شود.